

EMPLOYEE MANAGEMENT SYSTEM

Technical Documentation

Prepared by: Shivraj Mahatme

Technology: Spring Boot | Java | Oracle DB

Date: 15th June 2026

1. Project Overview

The Employee Management System is a RESTful web application developed using Spring Boot and Java 17. It provides a complete set of APIs to manage employee records within an organization, backed by an Oracle database.

1.1 Objective

To design and develop a backend application that enables organizations to perform full CRUD operations on employee data along with search and filter capabilities, proper validation, exception handling, and logging.

1.2 Technology Stack

Component	Technology
Language	Java 17
Framework	Spring Boot 3.5.15
Database	Oracle DB (FREEPDB1)
ORM	JPA / Hibernate 6.6
Build Tool	Maven
Logging	SLF4J + Logback (XML config)
Testing	JUnit 5 + Mockito 5
Server Port	9090

2. Assumptions

The following assumptions were made during the design and development of this application:

- Each employee has a unique system-generated ID (empId) assigned automatically via sequence strategy.
- Employee Code (empCode) is a business identifier provided at the time of creation and is not auto-generated.
- An employee's status must be one of three defined values: ACTIVE, INACTIVE, or RESIGNED.
- Salary is stored as a Long value representing the amount in the base currency (INR).
- Email validation is enforced at the API level; duplicate email handling is delegated to the database constraints.
- The application does not implement authentication or authorization; it assumes a secure internal network environment.
- Bulk save is supported — multiple employees can be saved in a single POST request.
- Soft delete is not implemented; employee records are permanently removed on delete.
- The Oracle database schema and user (SpringDB) are pre-configured and accessible at the provided JDBC URL.

3. Database Design

3.1 Table: EmployeeManagement

The application uses a single table named EmployeeManagement in the Oracle database. JPA/Hibernate manages the schema automatically via the `ddl-auto=update` configuration.

Column Name	Data Type	Constraints	Description
EmployeeID	NUMBER	PRIMARY KEY, NOT NULL	Auto-generated unique identifier
Name	VARCHAR2	NOT NULL	Full name of the employee
Email	VARCHAR2	NOT NULL	Valid email address of the employee
EmployeeCode	VARCHAR2	NOT NULL	Business-assigned employee code
Designation	VARCHAR2	NOT NULL	Job title or role of the employee
Salary	NUMBER	NOT NULL	Monthly salary in INR
Status	VARCHAR2	NOT NULL	One of: ACTIVE, INACTIVE, RESIGNED

3.2 Primary Key Strategy

The `empld` field uses `GenerationType.SEQUENCE`, which instructs Hibernate to use an Oracle sequence to generate unique IDs. This is the recommended strategy for Oracle databases as it is performant and avoids table-level locking.

3.3 Enum Mapping

The `status` field is mapped using `@Enumerated(EnumType.STRING)`, which stores the enum value as a readable string (e.g., 'ACTIVE') rather than an integer index. This improves data readability directly in the database.

3.4 Database Configuration

```
spring.datasource.url=jdbc:oracle:thin:@10.10.11.111:1521/FREEPDB1
spring.datasource.username=SpringDB
spring.datasource.password=*****
spring.jpa.hibernate.ddl-auto=update
server.port=9090
```

4. Entity Relationship Explanation

4.1 Overview

The current version of the Employee Management System contains a single entity: Employee. As there is only one table, there are no inter-table relationships (foreign keys or joins) in this version.

4.2 Employee Entity

Attribute	Java Type	JPA Annotation	Validation
empId	Long	@Id, @GeneratedValue	Auto-generated
empName	String	@Column(Name)	@NotBlank
email	String	@Column(Email)	@NotBlank, @Email
empCode	String	@Column(EmployeeCode)	@NotBlank
designation	String	@Column(Designation)	@NotBlank
salary	Long	@Column(Salary)	@NotNull
status	empStatus	@Enumerated(STRING)	@NotNull

4.3 Enum: empStatus

The empStatus enum defines the lifecycle of an employee within the organization:

Value	Meaning
ACTIVE	Employee is currently working in the organization
INACTIVE	Employee is temporarily not working (e.g., on leave)
RESIGNED	Employee has formally resigned from the organization

4.4 Future Scope for Relationships

- Department entity can be introduced with a Many-to-One relationship (many employees belong to one department).
- A Manager entity or self-referential Employee relationship can model reporting hierarchies.
- Leave Management entity can have a One-to-Many relationship with Employee.

5. Application Architecture

5.1 Layered Architecture

The application follows the standard Spring Boot layered architecture pattern:

Layer	Class	Responsibility
Controller	EmployeeController	Receives HTTP requests, validates input, returns HTTP responses
Service	employeeService	Contains business logic, logging, and exception handling
Repository	EmpRepo	Interfaces with the Oracle database via Spring Data JPA
Entity	Employee	Maps to the EmployeeManagement database table
Exception	EmployeeNotFoundException	Custom runtime exception for missing records
ExceptionHandler	GlobalExceptionHandler	Centralized exception handling via @ControllerAdvice

5.2 Request Flow

- Client sends an HTTP request to a Controller endpoint.
- Controller validates the request body using @Valid annotations.
- If validation fails, GlobalExceptionHandler returns a 400 BAD REQUEST with field-level error messages.
- Controller calls the appropriate Service method.
- Service logs the operation, applies business logic, and calls the Repository.
- If the record is not found, Service throws EmployeeNotFoundException.
- GlobalExceptionHandler catches it and returns 404 NOT FOUND.
- On success, Service returns the result to Controller, which wraps it in a ResponseEntity.

6. REST API Reference

6.1 Base URL

```
http://localhost:9090/employee
```

6.2 API Endpoints

Method	Endpoint	Description	Success Code
GET	/test	Health check	200 OK
POST	/save	Save one or more employees	201 CREATED
GET	/view	Retrieve all employees	200 OK
GET	/view/{employeeId}	Retrieve employee by ID	200 OK
PUT	/update/{employeeId}	Update employee by ID	200 OK
DELETE	/delete/{employeeId}	Delete employee by ID	200 OK
GET	/search/name?name=	Search employees by name	200 OK
GET	/search/designation?designation=	Search by designation	200 OK
GET	/search/status?status=	Filter by status	200 OK

6.3 Sample Request — POST /employee/save

```
[
  {
    "empName": "Rahul Sharma",
    "email": "rahul.sharma@company.com",
    "empCode": "EMP001",
    "designation": "Software Developer",
    "salary": 55000,
    "status": "ACTIVE"
  }
]
```

6.4 Sample Response — GET /employee/view/{id}

```
{
  "empId": 1,
  "empName": "Rahul Sharma",
  "email": "rahul.sharma@company.com",
  "empCode": "EMP001",
  "designation": "Software Developer",
  "salary": 55000,
  "status": "ACTIVE"
}
```

```
}
```

6.5 Error Responses

Scenario	HTTP Status	Response Body
Employee not found by ID	404 NOT FOUND	Employee not found with ID: {id}
Validation failure (blank name)	400 BAD REQUEST	{"empName": "Name cannot be blank"}
Invalid email format	400 BAD REQUEST	{"email": "Email must be valid format"}
Unexpected server error	500 INTERNAL SERVER ERROR	Something went wrong: {message}

7. Exception Handling

All exceptions are handled centrally in `GlobalExceptionHandler` using the `@ControllerAdvice` annotation. This ensures consistent error responses across all endpoints.

Exception Type	Handler Method	HTTP Status
<code>EmployeeNotFoundException</code>	<code>handleEmployeeNotFound()</code>	404 NOT FOUND
<code>MethodArgumentNotValidException</code>	<code>handleValidationErrors()</code>	400 BAD REQUEST
<code>IllegalArgumentException</code>	<code>handleIllegalArgument()</code>	400 BAD REQUEST
<code>Exception (generic)</code>	<code>handleGenericException()</code>	500 INTERNAL SERVER ERROR

8. Logging

8.1 Configuration

Logging is configured via `src/main/resources/logback.xml` using the Logback framework with SLF4J as the facade.

8.2 Log Outputs

Appender	Destination	Purpose
CONSOLE	IntelliJ / Terminal	Real-time log output during development
FILE (Rolling)	logs/employee-management.log	Persistent logs, rotated daily, kept for 30 days

8.3 Log Levels Used

Level	Where Used	Example
INFO	Normal operations	Saving 2 employee(s), Employees saved successfully
WARN	Validation failures	Validation failed: {empName: Name cannot be blank}
ERROR	Exceptions, not found	Employee not found with ID: 99
DEBUG	Spring Boot internals	Running with Spring Boot v3.5.15

9. Unit Testing

9.1 Framework

Unit tests are written using JUnit 5 and Mockito 5, both included via the spring-boot-starter-test dependency.

9.2 Test Coverage

Test Class	Layer	Methods Tested
EmployeeServiceTest	Service	testSaveEmployee(), testViewAllEmployee()
EmployeeControllerTest	Controller	testSaveEmployee(), testViewAll()

9.3 Key Annotations Used

Annotation	Purpose
@ExtendWith(MockitoExtension.class)	Enables Mockito in JUnit 5 tests
@Mock	Creates a mock (fake) instance of EmpRepo
@InjectMocks	Injects mock dependencies into the Service
@WebMvcTest	Loads only the Controller layer for testing
@BeforeEach	Sets up sample Employee data before each test
verify()	Confirms a specific method was called
when().thenReturn()	Defines mock behavior
assertEquals()	Asserts expected vs actual values

9.4 Test Results

The image displays two separate IDE windows showing the execution of test classes in a Java Spring Boot application. The top window is titled 'TestController' and shows two tests passed: 'testSaveEmply' (1 sec 228 ms) and 'testViewAll()' (233 ms). The console output for 'TestController' includes a ASCII art logo, the text ':: Spring Boot :: (v3.5.15)', and several log messages from 'E.Controller.TestController' and 'E.Controller.EmployeeController'. The bottom window is titled 'TestService' and also shows two tests passed: 'testSaveEmply' (3 sec 596 ms) and 'testViewAllEmployee' (42 ms). The console output for 'TestService' includes the command 'C:\Users\shivraj.mahatme\.jdk\ms-17.0.19\bin\java.exe ...' and log messages from 'E.Service.employeeService'.

```
Run EmployeeManagementSystemApplication x TestController x

2 tests passed 2 tests total, 1 sec 461 ms
  ✓ testSaveEmply 1 sec 228 ms
  ✓ testViewAll() 233 ms

:: Spring Boot :: (v3.5.15)

2026-06-12 14:15:59 [main] INFO E.Controller.TestController - Starting TestController using Java 17.0.19 with PID
10152 (started by shivraj.mahatme in C:\Users\shivraj
.mahatme\Downloads\EmployeeManagementSystem\EmployeeManagementSystem)
2026-06-12 14:15:59 [main] DEBUG E.Controller.TestController - Running with Spring Boot v3.5.15, Spring v6.2.19
2026-06-12 14:15:59 [main] INFO E.Controller.TestController - No active profile set, falling back to 1 default
profile: "default"
OpenJDK 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has
been appended
2026-06-12 14:16:06 [main] INFO E.Controller.TestController - Started TestController in 7.694 seconds (process
running for 12.737)
2026-06-12 14:16:07 [main] INFO E.Controller.EmployeeController - POST /employee/save called with 1 record(s)
2026-06-12 14:16:07 [main] INFO E.Controller.EmployeeController - GET /employee/view called

Process finished with exit code 0

Structure Alt+7 yeeManagementSystemApplication x TestService x

2 tests passed 2 tests total, 3 sec 638 ms
  ✓ testSaveEmply 3 sec 596 ms
  ✓ testViewAllEmployee 42 ms

C:\Users\shivraj.mahatme\.jdk\ms-17.0.19\bin\java.exe ...
OpenJDK 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has
been appended
2026-06-12 14:17:20 [main] INFO E.Service.employeeService - Saving 1 employee(s)
2026-06-12 14:17:20 [main] INFO E.Service.employeeService - Employees saved successfully
2026-06-12 14:17:21 [main] INFO E.Service.employeeService - Fetching all employees
2026-06-12 14:17:21 [main] INFO E.Service.employeeService - Total employees found: 1

Process finished with exit code 0
```

10. README — Setup and Execution

10.1 Prerequisites

Requirement	Version
Java JDK	17 or above
Maven	3.6 or above
Oracle Database	FREEPDB1 (accessible at 10.10.11.111:1521)
IDE	IntelliJ IDEA
Postman	-

10.2 Postman Collection

Import the provided Postman collection file (EmployeeManagementSystem.postman_collection.json) into Postman to test all available APIs. Set the base URL variable to `http://localhost:9090`.

API	Method	URL
Health Check	GET	<code>http://localhost:9090/employee/test</code>
Save Employee(s)	POST	<code>http://localhost:9090/employee/save</code>
View All Employees	GET	<code>http://localhost:9090/employee/view</code>
View by ID	GET	<code>http://localhost:9090/employee/view/1</code>
Update Employee	PUT	<code>http://localhost:9090/employee/update/1</code>
Delete Employee	DELETE	<code>http://localhost:9090/employee/delete/1</code>
Search by Name	GET	<code>http://localhost:9090/employee/search/name?name=Rahul</code>
Search by Designation	GET	http://localhost:9090/employee/search/designation?designation=Java Developer
Filter by Status	GET	<code>http://localhost:9090/employee/search/status?status=ACTIVE</code>

Outputs:

API: `localhost:9090/employee/save`

The screenshot shows a REST client interface with the following details:

- URL:** `localhost:9090/employee/save`
- Method:** `POST`
- Body:** A JSON array with two employee objects:

```
[
  {
    "empName": "Rahul Kulkarni",
    "email": "rahul.kulkarni@example.com",
    "empCode": "EMP-MH-001",
    "designation": "Software Engineer",
    "salary": 65000,
    "status": "ACTIVE"
  },
  {
    "empName": "Priyanka Joshi",
    ...
  }
]
```
- Status:** 201 Created
- Time:** 676 ms
- Size:** 197 B
- Response:** `Employees Saved Successfully`

API: `localhost:9090/employee/view`

The screenshot shows a REST client interface with the following details:

- URL:** `localhost:9090/employee/view`
- Method:** `GET`
- Query Params:** A table with 3 columns: Key, Value, and Description.
- Status:** 200 OK
- Time:** 297 ms
- Size:** 6.21 KB
- Response:** A JSON array of employee objects:

```
[
  {
    "empId": 2002,
    "empName": "Rahul Kulkarni",
    "email": "rahul.kulkarni@example.com",
    "empCode": "EMP-MH-001",
    "designation": "Software Engineer",
    "salary": 65000,
    "status": "ACTIVE"
  },
  {
    "empId": 2003,
    "empName": "Priyanka Joshi",
    "email": "priyanka.joshi@example.com",
    "empCode": "EMP-MH-002",
    ...
  }
]
```

API: `localhost:9090/employee/view/2003`

Employee / ViewById

GET localhost:9090/employee/view/2003

Send

Params Authorization Headers (7) Body Pre-request Script Tests Settings Cookies

Query Params

Key	Value	Description	...	Bulk Edit
Key	Value	Description		

Body Cookies Headers (5) Test Results

Status: 200 OK Time: 16 ms Size: 331 B Save as example

Pretty Raw Preview Visualize JSON

```
1 {
2   "empId": 2003,
3   "empName": "Priyanka Joshi",
4   "email": "priyanka.joshi@example.com",
5   "empCode": "EMP-MH-002",
6   "designation": "Project Manager",
7   "salary": 120000,
8   "status": "ACTIVE"
9 }
```

API: localhost:9090/employee/update/1109

Employee / Update

PUT localhost:9090/employee/update/1109

Send

Params Authorization Headers (9) Body Pre-request Script Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON

Beautify

```
1 {
2   "empId": 1109,
3   "empName": "Manasi Jadhav",
4   "email": "manasi.jadhav@example.com",
5   "empCode": "EMP308",
6   "designation": "Java Developer",
7   "salary": 75000,
8   "status": "INACTIVE"
9 }
```

Body Cookies Headers (5) Test Results

Status: 200 OK Time: 36 ms Size: 325 B Save as example

Pretty Raw Preview Visualize JSON

```
1 {
2   "empId": 1109,
3   "empName": "Manasi Jadhav",
```

API: localhost:9090/employee/delete/2003

HTTP Employee / Delete

DELETE localhost:9090/employee/delete/2003

Send

Params Authorization Headers (9) Body • Pre-request Script Tests Settings Cookies

Query Params

Key	Value	Description	...	Bulk Edit
Key	Value	Description		

Body Cookies Headers (5) Test Results

Status: 200 OK Time: 39 ms Size: 207 B Save as example

Pretty Raw Preview Visualize Text

```
1 Employee with ID 2003 deleted successfully!
```

API: <http://localhost:9090/employee/search/name?name=Sachin Deshmukh>

HTTP Employee / Search-Name

GET http://localhost:9090/employee/search/name?name=Sachin Deshmukh

Send

Params • Authorization Headers (9) Body • Pre-request Script Tests Settings Cookies

Query Params

☒ Key	Value	Description	...	Bulk Edit
☒ name	Sachin Deshmukh			
Key	Value	Description		

Body Cookies Headers (5) Test Results

Status: 200 OK Time: 45 ms Size: 636 B Save as example

Pretty Raw Preview Visualize JSON

```
1 [
2   {
3     "empId": 2004,
4     "empName": "Sachin Deshmukh",
5     "email": "sachin.d@example.com",
6     "empCode": "EMP-MH-003",
7     "designation": "QA Analyst",
8     "salary": 50000,
9     "status": "ACTIVE"
10  },
11  ]
```